

强化学习第八次作业报告：DQN 算法

毛川

学号：2300013218

2025 年 11 月 18 日

摘要

本报告详细阐述了深度 Q 网络 (Deep Q-Network, DQN) 算法在两个典型强化学习环境——CartPole 和 Breakout 上的应用与实验结果。针对不同的任务复杂度，本实验设计了相应的网络架构、超参数配置和训练技巧，并对实验结果进行了深入的分析和讨论。

1. 实验一：CartPole 任务

1.1. DQN 算法实现细节

针对 CartPole 任务，DQN 算法的关键实现包括：

- 经验回放缓冲区 (Replay Buffer)**：在现有代码库中已实现经验回放缓冲区功能，用于存储 (s_t, a_t, r_t, s_{t+1}) 四元组，以打破数据间的相关性。
- 目标网络 (Target Network)**：实现周期性更新的目标网络机制。通过每执行 k 个 *episode* 后，将主网络 (Online Network) 的参数复制到目标网络，以稳定 Q 值估计的计算。

1.2. 引入的训练技巧

- 学习率衰减 (Learning Rate Decay)**：在训练过程中，采用逐渐降低学习率的策略，旨在精细化后期优化过程，提高模型的收敛性能。

1.3. 网络架构 (Network Architecture)

针对 CartPole 任务，我们设计了一个轻量级的前馈神经网络 (Feedforward Neural Network) 作为 Q 网络。输入状态为 $s_t \in \mathbb{R}^{\text{input_dim}}$ ，网络结构如下：

- FC1**：包含 64 个隐藏单元，采用 ReLU 激活函数。

- **FC2:** 输出层，维度等于动作空间大小，用于预测各动作的 Q 值。

所有线性层使用 **正态初始化** (Normal Initialization) 进行权重初始化，优化器采用 Adam。

1.4. 超参数配置

本实验使用的关键超参数配置如下：

```
{
  "buffer_size": 2000,
  "batch_size": 64,
  "episodes": 600,
  "copy_steps": 500,
  "lr": 0.001,
  "epsilon": 0.05,
  "gamma": 0.98
}
```

Listing 1: CartPole 任务超参数配置

1.5. 实验结果与分析

经过 600 个 episode 的训练，模型达到了最大奖励 200 分的满分表现。最终训练日志概览：

100%| | 600/600 [05:54<00:00, 1.69it/s, lr=5.29e-5, reward=200, step=72246]

1.5.1 实验分析

分析图 1 可知，在训练大约 200 个 episode 之后，模型的性能已趋于稳定，平均奖励 (Average Reward) 达到 200 分。这充分验证了 DQN 算法能够有效学习 CartPole 任务的控制策略。同时，快速收敛的现象表明该任务对于 DQN 而言相对简单。

1.5.2 扩展实验探索

我们进行了如下超参数调整尝试：

- **降低 Target 更新频率:** 将目标网络同步频率调整为每 500 个时间步 (step) 更新一次。结果显示，训练稳定性有所提高，但收敛速度相应减慢，如图 2。
- **降低 Buffer 大小:** 将经验回放缓冲区大小从 2000 降低至 800。初期收敛速度加快，原因是较差的早期经验被更快地替换。然而，训练后期模型的表现出现不稳定现象，如图 3。

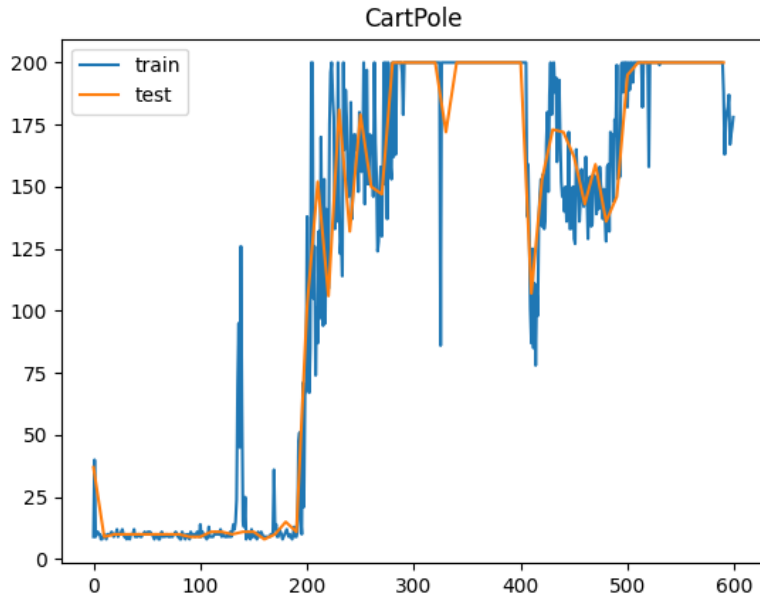


图 1: CartPole 任务训练曲线

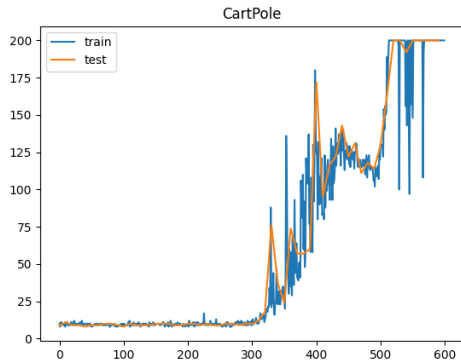


图 2: 降低 Target 更新频率

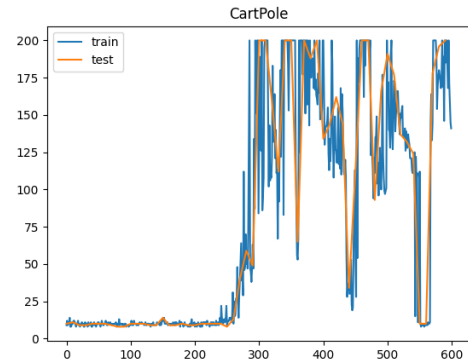


图 3: 降低 Buffer 大小

2. 实验二: Breakout 任务

2.1. DQN 算法与技巧

Breakout 任务的 DQN 基础实现与 CartPole 任务相同, 核心在于实现了经验回放缓冲区和目标网络机制。但针对 Atari 游戏的复杂性和高维状态空间, 我们在超参数和训练技巧上进行了大幅调整。

2.1.1 网络架构: Nature DQN

本实验采用经典的 **Nature DQN** 卷积神经网络结构, 用于处理图像输入。输入状态为 $s_t \in \mathbb{R}^{4 \times 84 \times 84}$, 网络结构如下:

1. **Conv1:** 32 个 8×8 卷积核, 步幅 (stride) 为 4, ReLU 激活。
2. **Conv2:** 64 个 4×4 卷积核, 步幅为 2, ReLU 激活。
3. **Conv3:** 64 个 3×3 卷积核, 步幅为 1, ReLU 激活。

卷积模块输出展平 (flatten) 后, 连接到两层全连接网络:

4. **FC1:** 512 个神经元, ReLU 激活。
5. **FC2:** 输出层, 维度等于动作空间大小, 输出对应状态 s_t 下所有可选动作的 Q 值 $Q(s_t, a)$ 。

2.1.2 数据预处理 (Preprocessing)

为提高计算效率和特征提取能力, 采用以下预处理步骤:

1. **高效处理:** 原始图像 ($210 \times 160 \times 3$) 经过灰度化、裁剪 (保留游戏区域) 和缩放, 最终尺寸降低为 ($84 \times 84 \times 1$)。
2. **堆叠历史帧 (Frame Stacking):** 将最近的 4 帧图像堆叠起来, 作为网络的输入, 形成形状为 ($84 \times 84 \times 4$) 的张量, 以捕捉动作的动态信息 (如小球的速度和方向)。
3. **手动射击 (Manual Fire):** 为了加快训练过程的启动, 我们在每个 *episode* 开始以及生命值减少时, 手动执行“FIRE”动作以继续游戏。

2.1.3 探索-利用策略与衰减计划

为了在训练过程中实现从“探索”到“利用”的平稳过渡, 我们采用了分段线性衰减的 ϵ -greedy 策略。

- **初始保持:** 在前 50,000 帧中, 维持最大探索率 $\epsilon = 1.0$ 。
- **第一阶段衰减:** 在接下来的 1,000,000 帧中, 将 ϵ 从 1.0 线性下降到 0.1。
- **第二阶段衰减:** 随后的 1,500,000 帧中, 继续将 ϵ 从 0.1 线性下降到最终的 0.01。
- **稳定阶段:** 完成衰减后, ϵ 固定为最小值 0.01, 保持稳定的利用行为。

2.2. 超参数配置

本实验使用的关键超参数配置如下:

```
{
  "buffer_size": 50000,
  "batch_size": 32,
  "episodes": 5000, % 实际上训练了 10000 个 episode
  "copy_steps": 10000,
  "stack_size": 4,
  "lr": 5e-5,
  "gamma": 0.98
}
```

Listing 2: Breakout 任务超参数配置

2.3. 实验结果

本实验共训练了 10,000 个 episode, 训练总时长约为 28 小时, 执行了约 5.1×10^6 个时间步。训练日志显示, DQN 算法处理复杂问题需要大量的训练时间和数据才能收敛。

部分训练日志示例:

35%| | 3501/10000 [3:43:48<26:49:37, epsilon=0.307, ...]

最终训练日志:

100%| | 10000/10000 [27:59:44<00:00, epsilon=0.01,
loss=0.015, reward=356, step=5176680]

最终模型在 Breakout 上的平均 *return* 达到了 356, 表现出色, 验证了 DQN 能够有效学习复杂的 Atari 游戏策略。

2.4. 实验分析与讨论

- 训练时间长:** 如图 4 所示, DQN 在高维、复杂环境中收敛需要大量的训练时间和数据。建议考虑使用更高效的算法, 如 Double DQN 或 Dueling DQN, 以提升学习效率和训练速度。
- 训练稳定性差:** DQN 训练过程中可能会出现不稳定现象, 导致性能波动较大。观察到模型性能在后期表现出“前一个 *episode* 得分很高 (350+), 后一个 *episode* 得分骤降 (5-)”的情况。对此进行分析:
 - 探索策略影响:** ϵ -greedy 策略在后期仍保留一定概率的随机探索, 可能导致偶尔选择不利动作。然而, 在测试阶段 (采用完全贪婪策略) 表现依然不稳定。

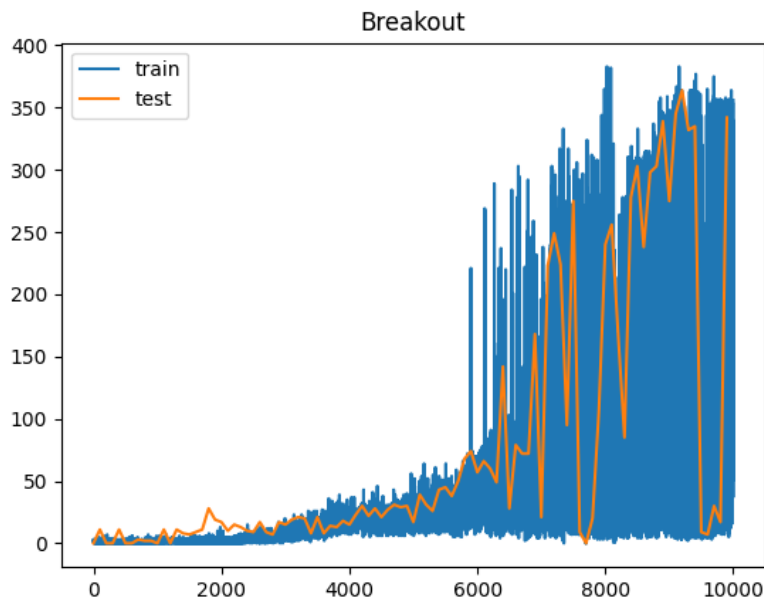


图 4: Breakout 任务训练曲线

- **模型崩溃与 Q 值估计:** 当得分较低时, 损失函数 (Loss) 值通常不大, 表明 Q 值估计并未出现严重偏差或模型训练崩溃。
- **游戏随机性:** 综上分析, 不稳定表现可能主要归因于 Breakout 游戏本身存在的较大随机性 (例如小球的快速弹射), 导致较优策略也难以应对所有困难情景, 从而影响模型的表现稳定性。

3. 未来的优化方向:

- 引入 **Double DQN** 机制以减少 Q 值过估计偏差, 加快收敛速度。
- 优化目标网络更新频率: 前期可采用较快的 Target Network 更新频率以加速学习, 后期适当降低更新频率以稳定训练。